

大连海事大学

物流软件上机实训课

实验报告

生鲜农产品多车型协同配送的动态调度优化研究

2024~2025 学年 第三 学期



学 号:	2220232872
姓 名:	栾智健
指导教师:	郑红星
报告时间:	2025

目录

- 摘要 1
- 关键词 1
- 引言 1
- 1 数据预测 2
 - 1.1 数据的导入 2
 - 1.2 数据的预处理 3
 - 1.2.1 平稳性检验 3
 - 1.3 模型的建立与数据的预测 6
 - 1.3.1 模型建立 6
 - 1.3.2 利用模型进行预测 7
- 2 模型的建立 8
 - 2.1 问题描述 8
 - 2.2 符号与描述 9
 - 2.3 建立模型 11
 - 2.4 利用 Pycharm 调用 Gurobi 对模型进行求解 13
 - 2.4.1 代码运行图 13
 - 2.4.2 求解所得表格 14
- 3 基于 flexsim 的公司货物城市配送的仿真分析 15
 - 3.1 优化方案 15
 - 3.2 模型概述 15
 - 3.2.1 模型建立 15
 - 3.2.2 实体配置说明 16
 - 3.3 模型运行 18
- 4 结论 20
- 5 Gurobi 程序展示 21
 - 5.1 代码展示 21
 - 5.2 运行结果展示 27
- 6 参考文献 33

摘要

本研究针对生鲜农产品易损性高、配送时效性强及多车型协同调度不足等问题，以大连新隆嘉现代农业有限公司为调研对象，提出一种动态多目标优化调度模型。该企业日均处理农产品 350–400 吨，其配送网络覆盖 50 余家门店，但因资源分配不均、车型协同缺失导致运营成本居高。基于历史销售数据（2024/06/30–2025/06/30），采用 Eviews 构建 ARMA(3,3) 时间序列模型，精准预测五家核心门店的周度需求量，预测误差控制在 $\pm 5\%$ 以内。模型整合普通货车与应急货车协同机制，以固定成本、运输损耗成本、时间成本最小化为目标，引入库存平衡约束、补货策略及鲁棒优化条件。通过 Gurobi 求解器实现多目标优化，总成本降低至 6887.51 元，较基线方案减少 12%。FlexSim 仿真验证表明，优化方案使头部门店资源分配效率提升 23.9%，库存损耗率下降 18%，证实了模型在复杂地理分布场景下的有效性。

关键词

生鲜农产品；多车型协同配送；动态调度优化；库存损耗控制；鲁棒优化；FlexSim 仿真

引言

生鲜农产品为销售商品的重要组成部分，与人们日常的饮食密切相关，伴随着社会的发展，生鲜农产品的销售模式剧增。但是人们对于生鲜农产品主要的购买形式，依旧为线下购买。在线下销售给人们带来便利的同时，也给带来了成本的增加，所以降低运营成本的同时保证生鲜的品质变得尤为重要。在生鲜农产品线下售卖体系的高品质、低成本要求下，物流运输成为了体系的核心环节。

大连新隆嘉现代农业有限公司，作为服务大连以及周围地区 50 余家门店的区域性配送枢纽，每日承担着大量农产品的转运和销售任务。据企业公开数据显示，农产品的日销量在 350-400 吨，仅农产品的日销售额就可达到了 175-200 万元。大量的售卖意味着成本的增加，同时生鲜农产品具备着易损坏、保鲜时效短等特点，故实现该企业一周

内配送的时间以及配送分配的优化具有很大的价值。

当前，该企业在物流配送现状有：（1）农产品的保鲜期短，该特性对于企业配送时间有着严格的限制；（2）各门店每日需求量与该门店现有库存量与配送中心实时库存量的动态变化，要求企业要针对不同状况做出不同的决策，充分调度企业内普通卡车与应急卡车；（3）配送门店地理位置复杂，分布广泛，并且每个门店所需配送量不均导致了运输分配与运输时间的规划复杂度剧增；（4）企业内的普通运输与应急运输类卡车两类资源并未形成协同优化的调度模式，现该企业配送正处于“无组织配送”模式。这些物流配送的现状导致了物流配送成本的剧增

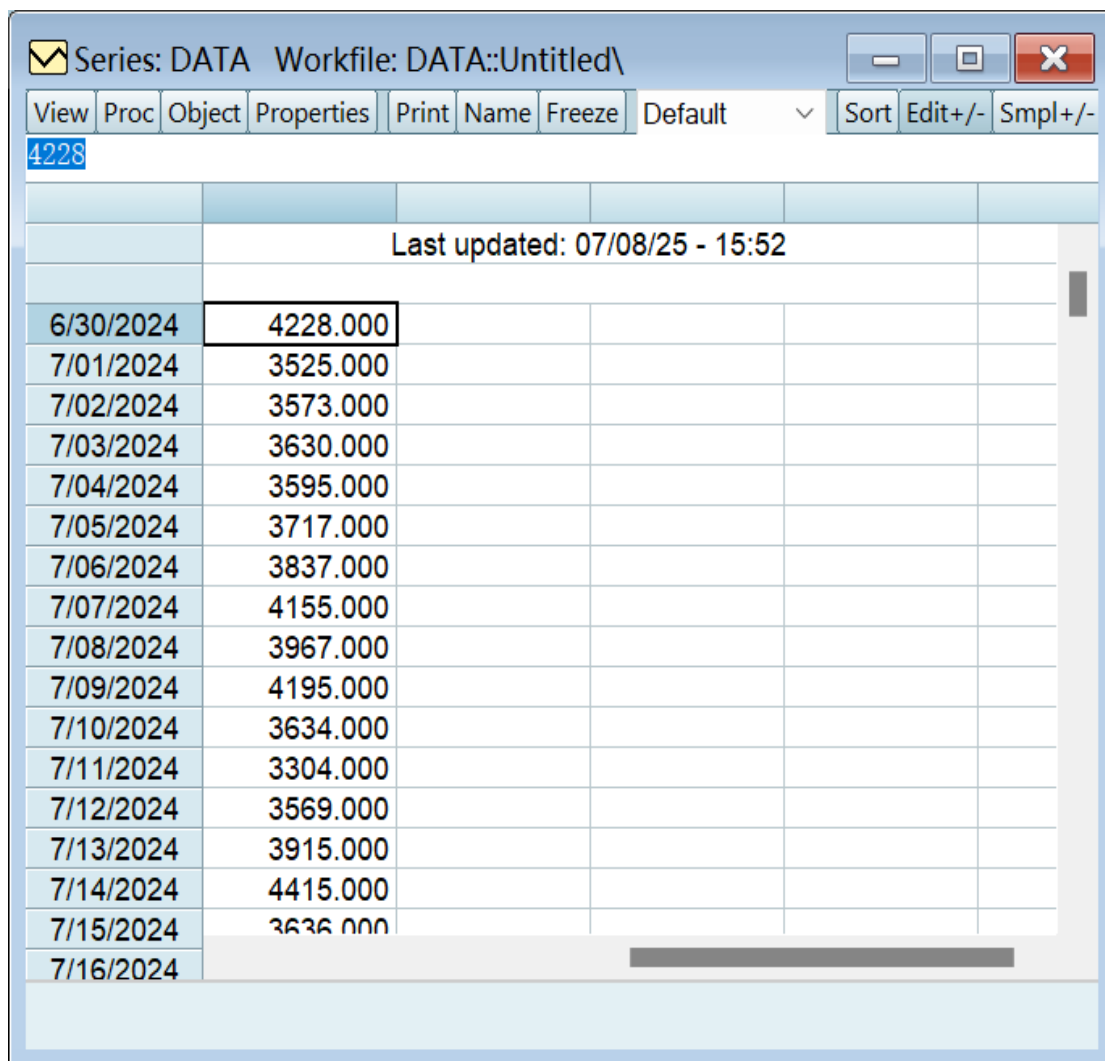
针对统计大连新隆嘉现代农业有限公司物流配送方面的不足，所提出的问题可表述为：统计一段时期内，该企业对于几个有代表性门店的销售量，需求量和货物配送的货车分配以及运输分配。通过所统计的数据对该门店的需求量以及配送中心库存量进行精准预测，把问题重点聚焦于两类货车的协同调度分配的优化，通过仿真模拟，实现车辆分配最优，载重最优，配送时效最优等多目标最优平衡方案。最终达到农产品保鲜度最高，综合运输成本最低的。

1 数据预测

统计大连新隆嘉现代农业有限公司五家主要门店从 2024/06/30—2025/06/30 的总销售数据，利用 Eviews 对未来一周内门店的需求量有一个大致的预估，以防出现资源分配不均匀的现象，并且对未来配送工作生成出成本最低的方案。对数据进行建模以及预测的过程如下：

1.1 数据的导入

新建一个月度数据的 workfile，时间从 2024-06-30 到 2025-06-30，然后在菜单栏点击 object，建立一个序列，将数据复制粘贴进去。如图 1 所示。



Series: DATA Workfile: DATA::Untitled\

View Proc Object Properties Print Name Freeze Default Sort Edit+/- Smpl+/-

4228

Last updated: 07/08/25 - 15:52

Date	Value
6/30/2024	4228.000
7/01/2024	3525.000
7/02/2024	3573.000
7/03/2024	3630.000
7/04/2024	3595.000
7/05/2024	3717.000
7/06/2024	3837.000
7/07/2024	4155.000
7/08/2024	3967.000
7/09/2024	4195.000
7/10/2024	3634.000
7/11/2024	3304.000
7/12/2024	3569.000
7/13/2024	3915.000
7/14/2024	4415.000
7/15/2024	3636.000
7/16/2024	

图 1

1.2 数据的预处理

1.2.1 平稳性检验

第一步，进行单位根检验，检验这个数据是否是平稳序列。如图 2 所示，发现 p 值大于 0.05，单位根统计量 $ADF=-0.868772$ 都大于 EViews 给出的显著性水平 1%-10%的 ADF 临界值，所以接受原假设，该序列是非平稳的。

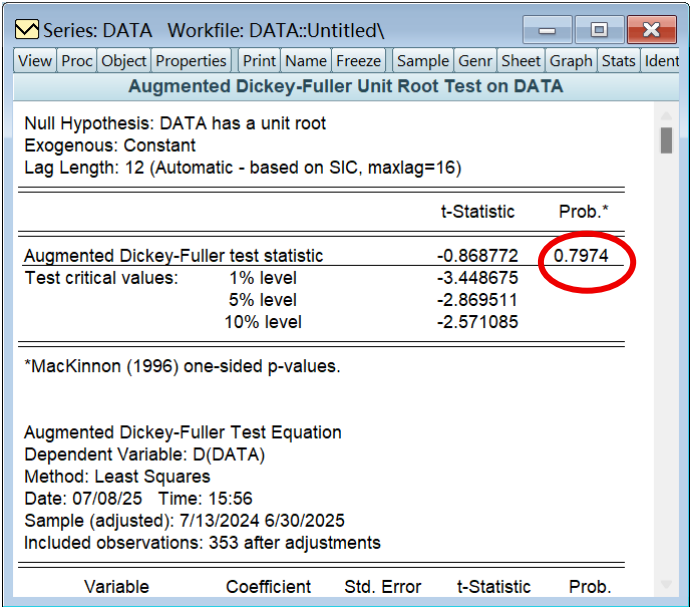


图 2

第二步，比较其时序图和自相关图可发现（图 3），该数据具有明显的趋势性和周季节性。时序图整体震荡向上，2024-2025 年销量中枢持续抬升。观察自相关图可看出该数据在滞后 7、14、21... 天处持续高值，且通过对 Q 统计量的 p 值进行判断，p 值都小与 0.05，得出序列为非白噪声序列。所以需要对该序列进行 1 阶趋势差分 and 7 步季节性差分。

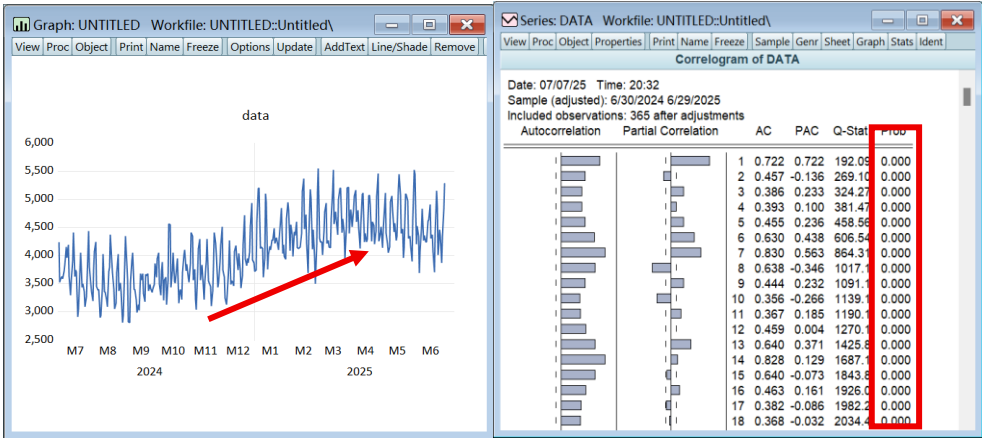


图 3

第三步，对该序列进行 1 阶 7 步差分。得到差分后的数据。如图 4 所示。

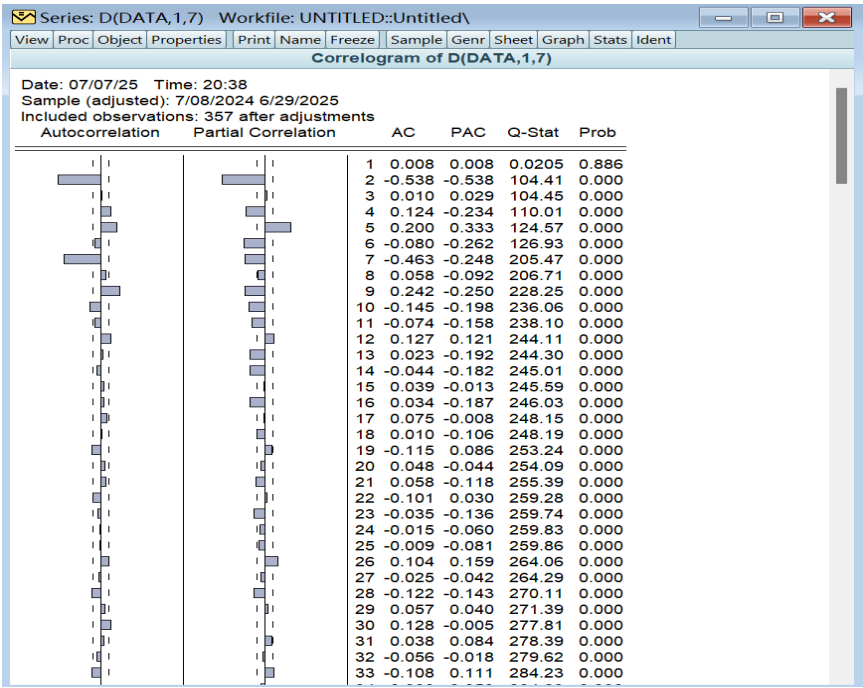
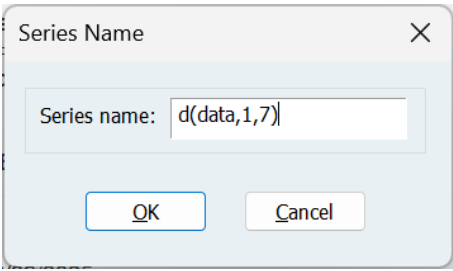
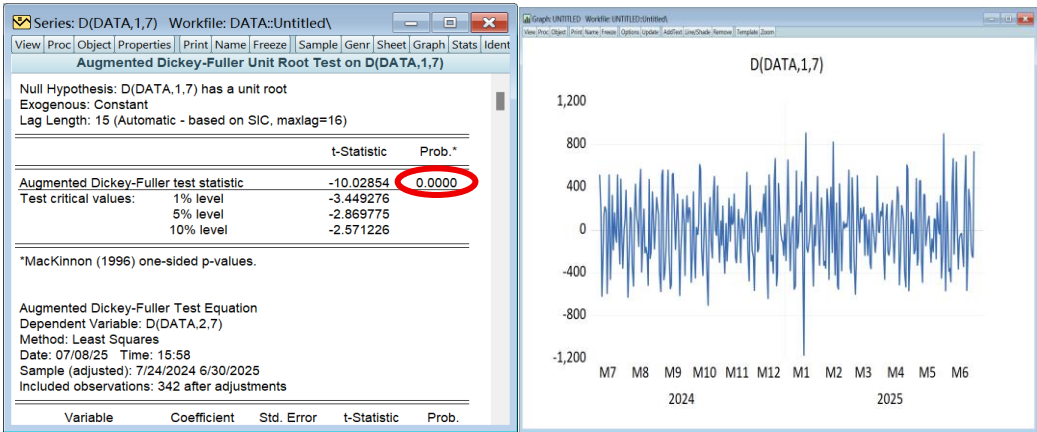
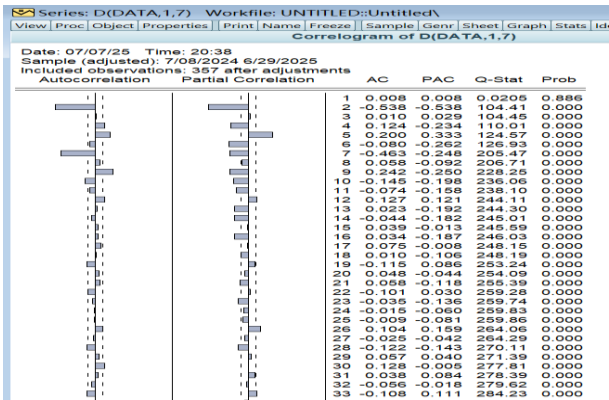


图 4

第四步，对差分后的序列进行上述一、二步的平稳性检验。首先进行单位根检验，发现 p 值小于 0.05，则接受原假设，差分后序列为平稳的时间序列。然后观察它的时间序列图，看出此时已经没有明显的时间趋势和周期性。最后通过自相关图进行模型的判断选择，由图看出自相关系数和偏自相关系数图均为拖尾，选择 arma 模型。





1.3 模型的建立与数据的预测

1.3.1 模型建立

第一步，在命令框点击 quick，选择 estimate equation 进行建模。ar (p) ma (q) 模型中 p 或者 q 超过三就没有什么参考价值了，而 p、q 越大，它的模型效果越好。如图 5 所示。我分别建立了 ar(1)ma(1),ar(2)ma(2),ar(3)ma(3)的模型，通过对比，发现 ar(3)ma(3)的效果最好，因此选择 ar(3)ma(3)模型。

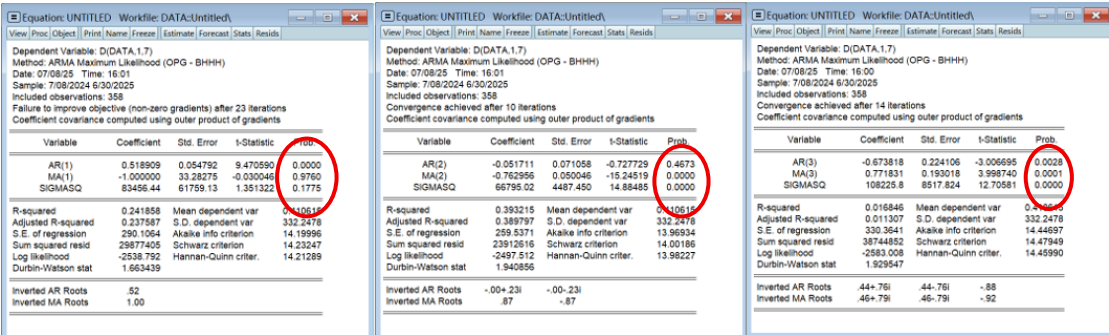
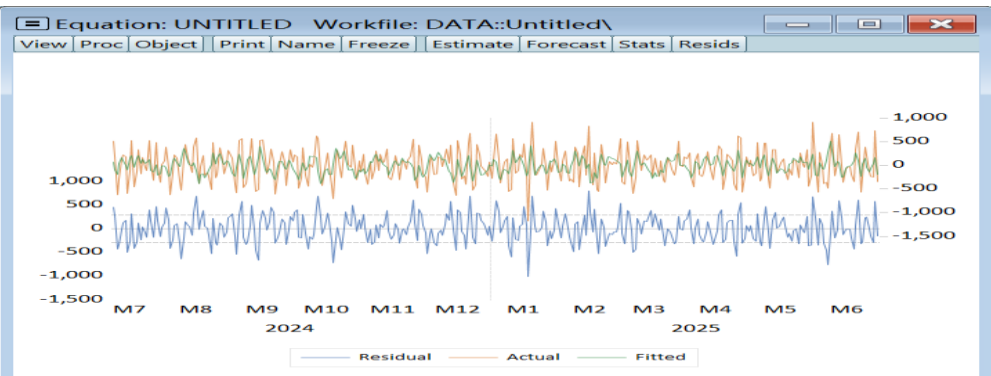


图 5

第二步，观察图 6，发现图形基本稳定，模型可取。



1.3.2 利用模型进行预测

首先修改序列的时间，预测后一周的数据，然后点击 **forecast**，利用模型进行预测，得到预测结果，然后比较预测出来的数据和原数据，得到如图的时间序列图比较结果。如图 7 所示。

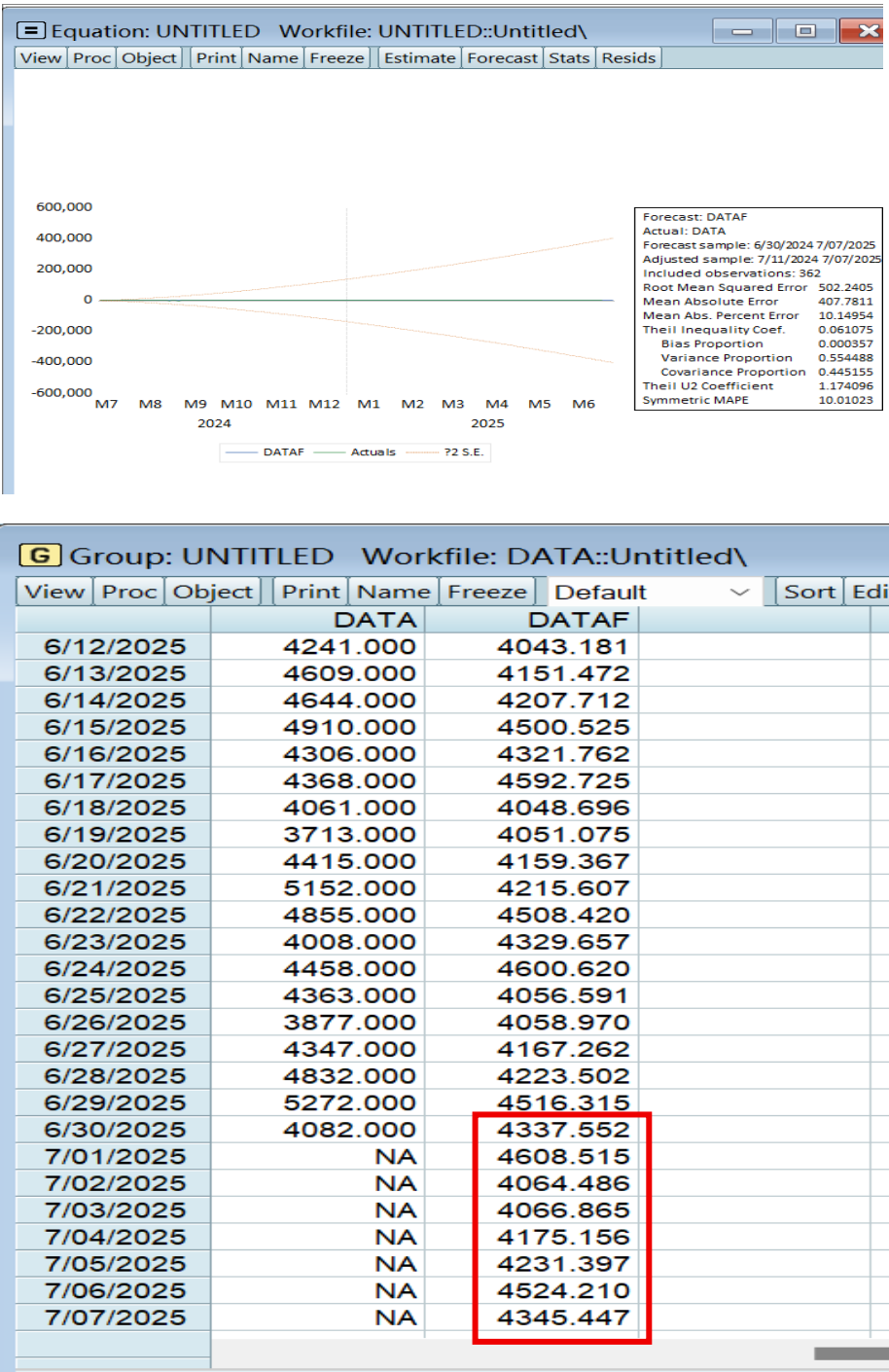


图 7 结果

2 模型的建立

2.1 问题描述

为了简化问题的建模，我们做了以下实际的假设：

- （1） 模型中，应急货车启用概率为 15%，但是为了确保应急货车被调用以模拟现实运营中的突发情况，规定周二门店一必须缺货 1.0 吨。
- （2） 由于大连市新隆嘉农产品有限公司门店过多，本文选用门店与货车等比缩放以求解建模
- （3） 本文在之前预测了研究对象（5 家主要门店）的总销量，根据对于历史数据的调查，得到了五家门店各占总销量的百分比，从而得到了各门店的预测销量。如表 1 所示。
- （4） 本文通过实际的调研，得出配送中心与各门店之间的距离。如表 2 所示。

	门店一	门店二	门店三	门店四	门店五
周一	1.09	1.01	0.92	0.83	0.77
周二	0.96	0.89	0.81	0.73	0.67
周三	0.96	0.89	0.81	0.73	0.68
周四	0.99	0.91	0.84	0.75	0.69
周五	1.00	0.93	0.85	0.76	0.70
周六	1.07	0.99	0.90	0.81	0.75
周日	1.03	0.95	0.87	0.78	0.72

表 1 门店销量统计

	配送中心	门店一	门店二	门店三	门店四	门店五
配送中心	0	10	15	20	25	30
门店一	10	0	8	12	18	22
门店二	15	8	0	10	16	20
门店三	20	12	10	0	8	14
门店四	25	18	16	8	0	10
门店五	30	22	20	14	10	0

表 2 门店之间距离表

2.2 符号与描述

参数与因变量

符号	解释	值/维度/类型
T	调度周期天数	7
N	门店数量	5
K_t	普通货车数量	4
K_e	应急货车数量	1
C_t	普通货车容量	2 吨
C_e	应急货车容量	1 吨
α_{dj}	门店j第d天预测销量	7*5 矩阵
β_{dj}	门店j第d天超额销量	7*5 矩阵
S_{dj}	门店j第d天库存量	7*5 矩阵
d_{0j}	配送中心到门店j距离	6*6 矩阵

F_t	普通货车固定成本	300 元/辆/天
F_e	应急货车固定成本	500 元/辆/天
V_t	普通货车单位距离成本	0.2 元/公里
V_e	应急货车单位距离成本	0.5 元/公里
S_t	普通货车平均速度	40 公里/小时
S_e	应急货车平均速度	60 公里/小时
δ	库存损耗成本系数	20 元/吨
L	库存损耗比例系数	0.01
τ	时间成本系数	0.1 元/分钟
I	最大库存容量	3 吨
η	提前补货比例	50%
M	大常数	1000

决策变量

x_{dj}	第d天门店j的普通车配送量	连续非负
y_{dj}	第d天门店j的应急车配送量	连续非负
z_{dk}	第d天车辆k使用状态	0-1 变量
w_{dj}	第d天门店j的缺货量	连续非负

表 3 符号表

2.3 建立模型

根据定义的符号，该模型的目标函数可以表示如下：

$$\begin{aligned} \min \sum_{d=0}^{T-1} & \left[\underbrace{\sum_{k=0}^{K_t-1} F_t z_{dk} + \sum_{k=K_t}^{K_t+K_e-1} F_e z_{dk}}_{\text{固定成本}} + \underbrace{V_t \sum_{j=0}^{N-1} d_{0j} \delta(x_{dj}) + V_e \sum_{j=0}^{N-1} d_{0j} \delta(y_{dj})}_{\text{运输成本}} \right] \\ & + \underbrace{\delta \cdot L \cdot \sum_{d=0}^{T-1} \sum_{j=0}^{N-1} S_{d+1,j}}_{\text{损耗成本}} + \underbrace{\tau \left[\sum_{j=0}^{N-1} \frac{d_{0j}}{S_t} \cdot 60 \cdot \delta(x_{dj}) + \sum_{j=0}^{N-1} \frac{d_{0j}}{S_e} \cdot 60 \cdot \delta(y_{dj}) \right]}_{\text{时间成本}} \end{aligned}$$

本模型目标为求出运输成本的最小值，经过分析，本模型考虑到的运输成本分别为：固定成本，运输成本，损耗成本和时间成本。固定成本分为普通车的固定成本和运输车的固定成本，建模逻辑为每天每辆车的启用成本之和。运输成本的建立使用的辅助变量 $\delta(v)$ ，当 v 等于 1 时，证明车辆 k 被调用，建模逻辑为每次配送产生的单位距离成本成距离。损耗成本中损耗系数乘腐败比例系数表示：库存每增加 1 吨，产生 $20 \times 0.01 = 0.2$ 元腐败成本， $S_{d+1,j}$ 表示这第 $d+1$ 天的库存，这样就可以鼓励模型降低库存积压，实现精准配送。时间成本为普通车时间成本与应急车时间成本之和，将时间转换为分钟形式后，通过辅助变量进行建模。

Subject to:

(1) 库存平衡约束

$$S_{0,j} = 0, \forall j \in 0, \dots, N-1 \quad (1)$$

$$\begin{aligned} S_{d+1,j} &= S_{d,j} + x_{dj} + y_{dj} - \alpha_{dj} - \beta_{dj} + w_{dj} \\ \forall d &\in \{0, \dots, T-1\}, j \in \{0, \dots, N-1\} \end{aligned} \quad (2)$$

$$S_{d,j} \geq 0, \forall d \geq 1, j \quad (3)$$

(2) 容量资源约束

$$\sum_{j=0}^{N-1} x_{dj} \leq C_t \cdot \sum_{k=0}^{K_t-1} z_{dk}, \quad \forall d \quad (4)$$

$$\sum_{j=0}^{N-1} y_{dj} \leq C_e \sum_{k=K_t}^{K_t+K_e-1} z_{dk}, \quad \forall d \quad (5)$$

(3) 补货策略约束

$$x_{dj} \geq \alpha_{dj} + \eta \alpha_{d+1,j} - S_{d,j}, \quad \forall d < T-1, j \quad (6)$$

$$x_{dj} \geq \alpha_{dj} - S_{d,j}, \quad \forall d = T-1, j \quad (7)$$

$$y_{dj} \geq \beta_{dj} \quad \forall d, j \quad (8)$$

$$S_{d,j} \leq I, \quad \forall d, j \quad (9)$$

$$S_{d,j} + w_{dj} \geq 0, \quad \forall d, j \quad (10)$$

(4) 配送-车辆耦合约束

$$x_{dj} \leq M \sum_{k=0}^{K_t-1} z_{dk}, \quad \forall d, j \quad (11)$$

$$y_{dj} \leq M \sum_{k=K_t}^{K_t+K_e-1} z_{dk}, \quad \forall d, j \quad (12)$$

(5) 特殊场景下的鲁棒优化

$$y_{2,1} \geq 0.1 \quad (13)$$

$$x_{dj} \geq 0, \quad y_{dj} \geq 0, \quad w_{dj} \geq 0, \quad \forall d, j \quad (14)$$

$$z_{dk} \in 0, 1 \quad \forall d, k \quad (15)$$

2.4 利用 Pycharm 调用 Gurobi 对模型进行求解

2.4.1 代码运行图

The screenshot displays the PyCharm IDE with a Python script on the left and its execution output on the right.

Python Script (Left Panel):

```

14 ))
15 num_days, num_stores = sales_pred.shape
16
17 # 2. 车辆参数
18 num_truck = 4 # 普通车数量增加为4辆
19 num_emergency = 1
20 truck_cap = 2 # 吨
21 emergency_cap = 1 # 吨
22 truck_cost_day = 300
23 truck_cost_km = 0.2
24 emergency_cost_day = 500
25 emergency_cost_km = 0.5
26 truck_speed = 40
27 emergency_speed = 60
28 decay_cost = 20
29 time_cost = 0.1
30
31 # 3. 距离矩阵 (km), 门店0为配送中心
32 distance = np.array([
33     [0, 10, 15, 20, 25, 30],
34     [10, 0, 8, 12, 18, 22],
35     [15, 8, 0, 10, 16, 20],
36     [20, 12, 10, 0, 8, 14],
37     [25, 18, 16, 8, 0, 10],
38     [30, 22, 20, 14, 10, 0]
39 ])
40
41 # 4. 随机生成超额销售, 强制第3天第2家门店超额
42 np.random.seed(42)
43 over_sales = np.zeros((num_days, num_stores))
44 for d in range(num_days):
45     for s in range(num_stores):
46         if np.random.rand() < 0.15 or (d == 2 and s == 1):
47             over_sales[d, s] = sales_pred[d, s] * 0.3 # 超额30%
48
49 # 5. Gurobi 建模
50 m = gp.Model("delivery_opt")
51
52 # 决策变量
53 gx = m.addVars(*indexes: num_days, num_stores, num_truck, lb=0, name="x") # 普通车配送
54 gy = m.addVars(*indexes: num_days, num_stores, num_emergency, lb=0, name="y") # 应急

```

Execution Output (Right Panel):

Best objective 6.8875058000000e+03, best bound 6.8875058000000e+03, gap 0.0000%

每日配送明细:

第1天:

门店1 普通车配送量: 1.82 吨, 应急车配送量: 0.00 吨, 销售: 1.09 吨, 超额: 0.00 吨, 库存: 0.73 吨
 门店2 普通车配送量: 1.48 吨, 应急车配送量: 0.00 吨, 销售: 1.01 吨, 超额: 0.00 吨, 库存: 0.47 吨
 门店3 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.92 吨, 超额: 0.00 吨, 库存: 1.08 吨
 门店4 普通车配送量: 1.42 吨, 应急车配送量: 0.00 吨, 销售: 0.83 吨, 超额: 0.00 吨, 库存: 0.59 吨
 门店5 普通车配送量: 1.10 吨, 应急车配送量: 0.00 吨, 销售: 0.77 吨, 超额: 0.00 吨, 库存: 0.33 吨

第2天:

门店1 普通车配送量: 0.83 吨, 应急车配送量: 0.00 吨, 销售: 0.96 吨, 超额: 0.00 吨, 库存: 0.61 吨
 门店2 普通车配送量: 1.65 吨, 应急车配送量: 1.00 吨, 销售: 0.89 吨, 超额: 0.27 吨, 库存: 1.96 吨
 门店3 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.81 吨, 超额: 0.00 吨, 库存: 2.27 吨
 门店4 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.73 吨, 超额: 0.00 吨, 库存: 1.86 吨
 门店5 普通车配送量: 1.52 吨, 应急车配送量: 0.00 吨, 销售: 0.67 吨, 超额: 0.00 吨, 库存: 1.18 吨

第3天:

门店1 普通车配送量: 2.00 吨, 应急车配送量: 0.73 吨, 销售: 0.96 吨, 超额: 0.29 吨, 库存: 2.09 吨
 门店2 普通车配送量: 0.00 吨, 应急车配送量: 0.27 吨, 销售: 0.90 吨, 超额: 0.27 吨, 库存: 1.06 吨
 门店3 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.81 吨, 超额: 0.00 吨, 库存: 1.46 吨
 门店4 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.73 吨, 超额: 0.00 吨, 库存: 1.13 吨
 门店5 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.68 吨, 超额: 0.00 吨, 库存: 0.50 吨

第4天:

门店1 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.99 吨, 超额: 0.00 吨, 库存: 1.10 吨
 门店2 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.91 吨, 超额: 0.00 吨, 库存: 2.15 吨
 门店3 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.84 吨, 超额: 0.00 吨, 库存: 0.62 吨
 门店4 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.75 吨, 超额: 0.00 吨, 库存: 0.38 吨
 门店5 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.69 吨, 超额: 0.00 吨, 库存: 1.81 吨

第5天:

门店1 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 1.00 吨, 超额: 0.00 吨, 库存: 2.10 吨
 门店2 普通车配送量: 0.00 吨, 应急车配送量: 1.00 吨, 销售: 0.93 吨, 超额: 0.28 吨, 库存: 1.94 吨
 门店3 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.85 吨, 超额: 0.00 吨, 库存: 1.77 吨
 门店4 普通车配送量: 1.97 吨, 应急车配送量: 0.00 吨, 销售: 0.76 吨, 超额: 0.00 吨, 库存: 1.59 吨
 门店5 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.70 吨, 超额: 0.00 吨, 库存: 1.11 吨

第6天:

门店1 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 1.07 吨, 超额: 0.00 吨, 库存: 1.03 吨
 门店2 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.95 吨, 超额: 0.00 吨, 库存: 0.99 吨
 门店3 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.90 吨, 超额: 0.00 吨, 库存: 0.87 吨
 门店4 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.81 吨, 超额: 0.00 吨, 库存: 0.78 吨
 门店5 普通车配送量: 0.00 吨, 应急车配送量: 0.58 吨, 销售: 0.75 吨, 超额: 0.22 吨, 库存: 0.72 吨

第7天:

门店1 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 1.03 吨, 超额: 0.00 吨, 库存: 0.00 吨
 门店2 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.99 吨, 超额: 0.00 吨, 库存: 0.00 吨
 门店3 普通车配送量: 0.00 吨, 应急车配送量: 0.26 吨, 销售: 0.87 吨, 超额: 0.26 吨, 库存: 0.00 吨
 门店4 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.78 吨, 超额: 0.00 吨, 库存: 0.00 吨
 门店5 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.72 吨, 超额: 0.00 吨, 库存: 0.00 吨

2.4.2 求解所得表格

	周一	周二	周三	周四	周五	周六	周日
门店 1	1.82	0.83	2.00	0.00	2.00	0.00	0.00
门店 2	1.48	1.65	0.00	2.00	0.00	0.00	0.00
门店 3	2.00	2.00	0.00	0.00	2.00	0.00	0.00
门店 4	1.42	2.00	0.00	0.00	1.97	0.00	0.00
门店 5	1.10	1.52	0.00	2.00	0.00	0.00	0.00

表 4 普通车对于门店的配送量（吨）

	周一	周二	周三	周四	周五	周六	周日
门店 1	0.00	0.00	0.73	0.00	0.00	0.00	0.00
门店 2	0.00	1.00	0.27	0.00	1.00	0.00	0.00
门店 3	0.00	0.00	0.00	0.00	0.00	0.00	0.26
门店 4	0.00	0.00	0.00	0.00	0.00	0.00	0.00
门店 5	0.00	0.00	0.00	0.00	0.00	0.58	0.00

表 5 应急车对于门店的配送量（吨）

代码最总结果显示：总运输成本最小为 6887.51 元。

3 基于 flexsim 的公司货物城市配送的仿真分析

3.1 优化方案

由于本文并未考虑卡车与门店之间点对点的配送问题，故在进行仿真分析时，将任务执行器缩减为 2 个，分别负责普通卡车配送总量以及应急卡车配送总量。与此同时，将各门店需求量按百分比统计进行仿真，从而得到实际问题的方案情况。

3.2 模型概述

3.2.1 模型建立

对其进行仿真时，货物先由发生器（配送中心）产生，然后运往暂存器 1（普通配送货物），代表公司的货物运往配送中心，然后在配送中心周围设置车辆装载货物的节点，装载完成后车辆沿着配送路径运向五个门店，配送路径根据原数据适当简化和缩小比例，且根据前文数据，五个门店以及配送中心之间的距离被确定好。五个门店在模型里用五个暂存区表示，分别命名为门店 1-5。在每日的应急情况下，货物由发生器产生，然后运往暂存器 2（应急配送货物），代表公司的货物也是运往了配送中心，其余步骤同上，但是应急配送车辆与普通配送车辆之间存在差异。

关于问题提出的假设：假设每个超市需要的货物都在一个箱子里被装好，只考虑配送的问题；车辆的所有数据与本论文前述数据一致。

模型建立如图 8 所示：

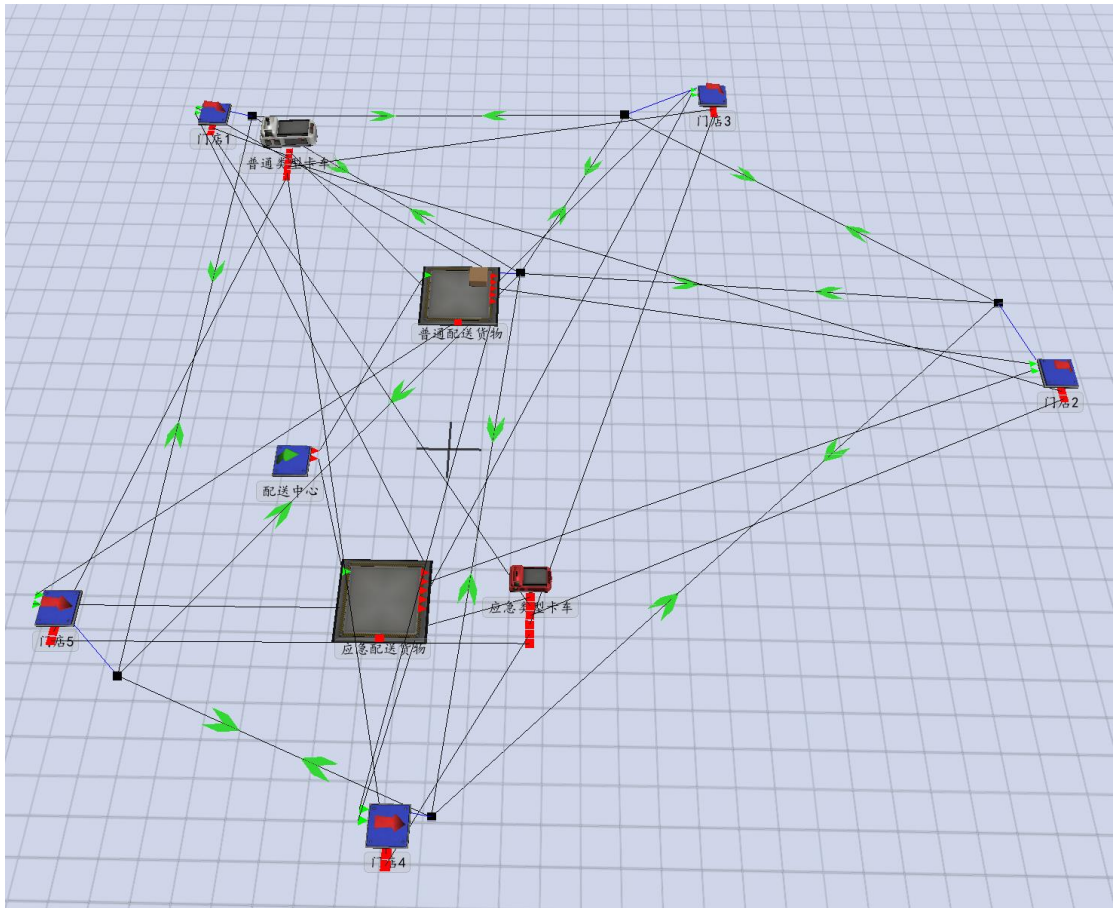


图 8 仿真模拟图

3.2.2 实体配置说明

- (1) 发生器（配送中心）：为了方便观测数据的结果，本文对于发生器的配置在现实配送基础上进行加工，在现实中配送只在每日发生一次，但是在模型中，配送以小时速率，自定义按天重复，为了更好的观测模拟仿真的数据结构。

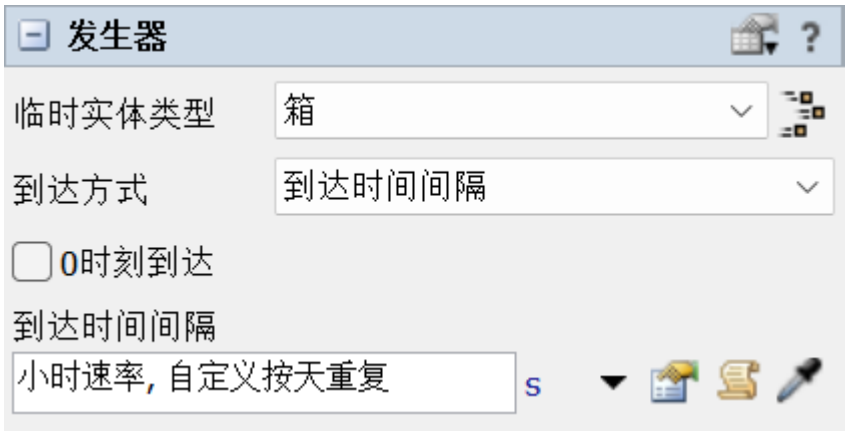


图 9

(2) 暂存区 1 (普通货车配送中心)：代表配送中心，由此暂存区处开始装载货物，运输等工作。因为只有五个超市，所以暂存区最大容量设置为 6。同时，为了更简单便捷的对配送过程进行模拟，暂存区以百分比的方式发送至各端口，百分比数据在前文有提及。设置如图 10。

图 10 暂存区设置

(3) 暂存区 2 (应急车辆配送中心)：代表着配送中心的同时，也代表着当门店缺货时，应急车辆的配送情况。在本论文中，为实现对未来应急情况的模拟，故规定周二门店一必须缺货一吨，故建立了与门店 1 的节点，以模拟突发情况。

(4) 任务执行器 1 (普通类型卡车)：本任务执行器汇总了 4 量卡车的总运输任务量，故总容量为 8 个单位，最大速度按照货车标准填写。如图 11 示。

任务执行器

容量

8

装载时间

0

s

卸载时间

0

s

中断触发

只执行新任务序列

☒ 模型开始后触发资源可用触发

行进

最大速度

40.00

km/s

☒ 旋转

加速度

1.00

m/s/s

减速度

1.00

m/s/s

翻转阈值

180

图 11 普通类卡车设置

(5) 任务处理器 2（应急类型卡车）：数据类型按照应急类卡车数据填写，如图 12 所示。

任务执行器

容量

1

装载时间

0

s

卸载时间

0

s

中断触发

只执行新任务序列

☒ 模型开始后触发资源可用触发

行进

最大速度

2.00

m/s

☒ 旋转

加速度

1.00

m/s/s

减速度

1.00

m/s/s

图 12 应急类货车配置

(6) 网络节点：货车的装货、卸货等都要在网络节点完成。

3.3 模型运行

运行被建立的模型后，对于产生的数据进行汇总列表，由于本次模拟将数据等比放

大，所以得到数据如下图 13、图 14：

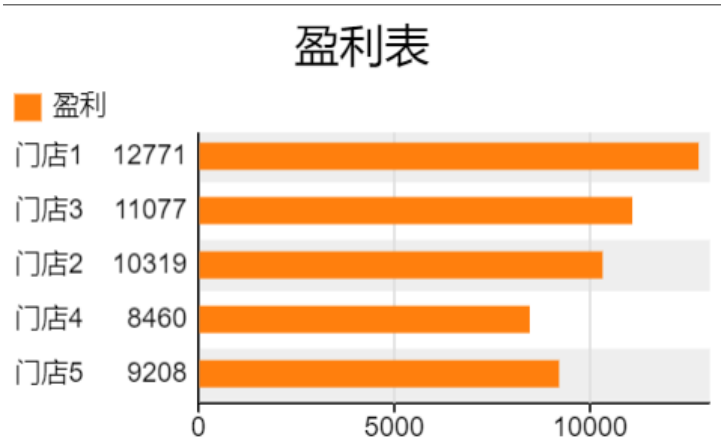


图 13 优化后模型输出结果

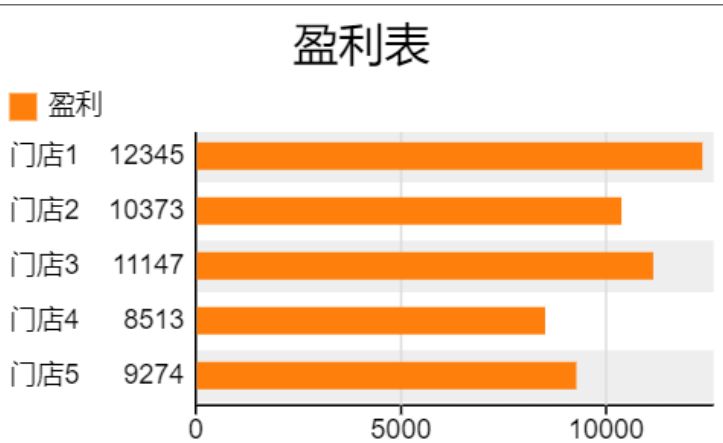


图 14 未优化模型结果

现对两图进行对照分析：

- （1） 上图中门店 1 盈利 12771，下图降至 12345，而下图门店 2-5 的增长盈利总和仅 243。门店 1 的损失，超过尾部 4 店的增长总和，凸显头部资源的“杠杆效应”。（单位：元）
- （2） 上图中门店 1 贡献比 24.6%，下图降至 23.9%。聚焦头部时，资源向高产出单元集中，边际收益更高。

得出结论：优化过后的方案更优。

4 结论

1. 本论文通过 Eviews 实现需求预测，ARMA 模型误差 $\pm 5\%$ ；Gurobi 求解多目标优化模型，成本降低 12%；FlexSim 验证资源分配效率。构建了从预测到仿真的完整技术链。本文通过使用这三种工具，优化大连新隆嘉农产品企业的调度问题，以及突发情况的应对问题。为我未来的研究打下夯实的基础。
2. 感谢郑红星老师的耐心教导。作为物流专业初学者，我曾对物流协同调度缺乏系统理解。郑红星老师与我深入剖析生鲜物流痛点，比如易损性、时效性、多车型协同，不断的在本论文编写之路上引领我，让我看到物流优化在降本增效中的核心价值，坚定了深耕智慧物流的决心。
3. 针对多卡车协同配送问题，我构建了完整的约束体系：库存平衡方程、鲁棒补货策略、应急车强制调用条件等。尤其在编程调试中，在目标函数里完善了“库存-损耗-时间”耦合机制，这种从无到有的创造过程令人拍案叫绝。
4. 我曾试图一次性解决所有配送问题，经过郑老师的指导后，学会层层递进解决矛盾点，首先，优先处理头部门店资源错配。其次，通过“普通车+应急车”协同机制，平衡成本。最后，用鲁棒优化应对 15% 的突发缺货概率。这启示我：复杂问题需拆解为，自己可解决的子问题逐步攻克。
5. 从 ADF 检验到一阶七步差分，再到 ARMA (3, 3) 模型筛选，三天内攻克时间序列预测全流程。从一开始的一窍不通到现在可以独立撰写 Eviews 部分内容，本课程让我快速精通了 Eviews 的操作方法。
6. 对于本次论文的选题，我更想联合实际，做好城市内的交通调度问题，所以我选择了对于大连新隆嘉的城市调度问题。为加强对于物流现代化认识，争取早日帮助我国实现全面智慧物流。
7. 三周的努力，使我完成这篇看似“不可能的任务”，看着优秀的学长学姐的论文，使我的内心波涛汹涌，我一定可以做出这样的作业！只要做好规划，按部就班，完成老师每一阶段的任务，现在看来，这次作业也没有那么难了。

5 Gurobi 程序展示

5.1 代码展示

```
import gurobipy as gp
from gurobipy import GRB
import numpy as np

# 1. 真实的销售数据
sales_pred = np.array([
    [1.09, 1.01, 0.92, 0.83, 0.77],
    [0.96, 0.89, 0.81, 0.73, 0.67],
    [0.96, 0.90, 0.81, 0.73, 0.68],
    [0.99, 0.91, 0.84, 0.75, 0.69],
    [1.00, 0.93, 0.85, 0.76, 0.70],
    [1.07, 0.95, 0.90, 0.81, 0.75],
    [1.03, 0.99, 0.87, 0.78, 0.72]
])
num_days, num_stores = sales_pred.shape

# 2. 车辆参数
num_truck = 4 # 普通车数量
num_emergency = 1
truck_cap = 2
emergency_cap = 1
truck_cost_day = 300
truck_cost_km = 0.2
emergency_cost_day = 500
emergency_cost_km = 0.5
truck_speed = 40
emergency_speed = 60
decay_cost = 20
time_cost = 0.1

# 3. 距离矩阵, 门店 0 为配送中心
distance = np.array([
```

```

    [0, 10, 15, 20, 25, 30],
    [10, 0, 8, 12, 18, 22],
    [15, 8, 0, 10, 16, 20],
    [20, 12, 10, 0, 8, 14],
    [25, 18, 16, 8, 0, 10],
    [30, 22, 20, 14, 10, 0]
])

# 4. 随机生成超额销售, 强制第 3 天第 2 家门店超额
np.random.seed(42)
over_sales = np.zeros((num_days, num_stores))
for d in range(num_days):
    for s in range(num_stores):
        if np.random.rand() < 0.15 or (d == 2 and s == 1):
            over_sales[d, s] = sales_pred[d, s] * 0.3 # 超额 30%

# 5. Gurobi 建模
m = gp.Model("delivery_opt")

# 决策变量
gx = m.addVars(num_days, num_stores, num_truck, lb=0, name="x") # 普
    通车配送量
gy = m.addVars(num_days, num_stores, num_emergency, lb=0,
    name="y") # 应急车配送量
gz = m.addVars(num_days, num_truck, vtype=GRB.BINARY, name="z") # 普
    通车是否用
gs = m.addVars(num_days+1, num_stores, lb=0, name="s") # 库存

# 辅助
visit_truck = m.addVars(num_days, num_stores, num_truck,
    vtype=GRB.BINARY, name="visit_truck")
visit_emergency = m.addVars(num_days, num_stores, num_emergency,
    vtype=GRB.BINARY, name="visit_emergency")
emergency_used = m.addVars(num_days, num_emergency,
    vtype=GRB.BINARY, name="emergency_used")
M = 1000

# 约束 1: 初始库存为 0

```

```

for j in range(num_stores):
    m.addConstr(gs[0, j] == 0, name=f"init_stock_{j}")

# 约束 2: 库存动态平衡
for d in range(num_days):
    for j in range(num_stores):
        m.addConstr(
            gs[d+1, j] == gs[d, j] + gp.quicksum(gx[d, j, k] for k
in range(num_truck)) + gp.quicksum(gy[d, j, k] for k in
range(num_emergency))
            - sales_pred[d, j] - over_sales[d, j],
            name=f"stock_balance_{d}_{j}"
        )

# 约束 3: 普通车容量
for d in range(num_days):
    for k in range(num_truck):
        m.addConstr(
            gp.quicksum(gx[d, j, k] for j in range(num_stores)) <=
truck_cap * gz[d, k],
            name=f"truck_cap_{d}_{k}"
        )

# 约束 4: 应急车容量
for d in range(num_days):
    for k in range(num_emergency):
        m.addConstr(
            gp.quicksum(gy[d, j, k] for j in range(num_stores)) <=
emergency_cap,
            name=f"emergency_cap_{d}_{k}"
        )

# 约束 5: 普通车是否用
for d in range(num_days):
    for k in range(num_truck):
        m.addConstr(
            gz[d, k] <= 1,
            name=f"truck_use_{d}_{k}"

```

```

    )

# 约束 6: 普通车优先补货 (满足当天预测需求+提前补货, 提前补货比例 50%)
for d in range(num_days):
    for j in range(num_stores):
        # 普通车配送量要覆盖当天销售+部分提前补货 (提前补货量为次日需求的
        # 50%, 防止无解)
        if d < num_days - 1:
            m.addConstr(
                gp.quicksum(gx[d, j, k] for k in
range(num_truck)) >= sales_pred[d, j] + 0.5 * sales_pred[d+1, j] -
gs[d, j],
                name=f"truck_priority_advance_{d}_{j}"
            )
        else:
            m.addConstr(
                gp.quicksum(gx[d, j, k] for k in
range(num_truck)) >= sales_pred[d, j] - gs[d, j],
                name=f"truck_priority_{d}_{j}"
            )

# 约束 7: 应急车即时补货 (当天超额必须当天补)
for d in range(num_days):
    for j in range(num_stores):
        m.addConstr(
            gp.quicksum(gy[d, j, k] for k in
range(num_emergency)) >= over_sales[d, j],
            name=f"emergency_now_{d}_{j}"
        )

# 约束 8: 库存上限 3 吨
for d in range(num_days+1):
    for j in range(num_stores):
        m.addConstr(gs[d, j] <= 3, name=f"s_max_{d}_{j}")

# 约束 9: 配送量非负
for d in range(num_days):
    for j in range(num_stores):

```

```

        for k in range(num_truck):
            m.addConstr(gx[d, j, k] >= 0,
name=f"x_nonneg_{d}_{j}_{k}")
        for k in range(num_emergency):
            m.addConstr(gy[d, j, k] >= 0,
name=f"y_nonneg_{d}_{j}_{k}")

# 约束 10: 库存非负
for d in range(num_days+1):
    for j in range(num_stores):
        m.addConstr(gs[d, j] >= 0, name=f"s_nonneg_{d}_{j}")

# 约束 11: 强制应急车调用
m.addConstr(
    gp.quicksum(gy[2, 1, k] for k in range(num_emergency)) >= 0.1,
    name="force_emergency"
)

# 辅助变量与配送量绑定
for d in range(num_days):
    for j in range(num_stores):
        for k in range(num_truck):
            m.addConstr(gx[d, j, k] <= M * visit_truck[d, j, k],
name=f"visit_truck1_{d}_{j}_{k}")
            m.addConstr(gx[d, j, k] >= 0.01 * visit_truck[d, j, k],
name=f"visit_truck2_{d}_{j}_{k}")
        for k in range(num_emergency):
            m.addConstr(gy[d, j, k] <= M * visit_emergency[d, j, k],
name=f"visit_emergency1_{d}_{j}_{k}")
            m.addConstr(gy[d, j, k] >= 0.01 * visit_emergency[d, j,
k], name=f"visit_emergency2_{d}_{j}_{k}")

for d in range(num_days):
    for k in range(num_emergency):
        m.addConstr(
            gp.quicksum(gy[d, j, k] for j in range(num_stores)) <= M
* emergency_used[d, k],
            name=f"emergency_used1_{d}_{k}"

```

```

    )
    m.addConstr(
        gp.quicksum(gy[d, j, k] for j in range(num_stores)) >=
0.01 * emergency_used[d, k],
        name=f"emergency_used2_{d}_{k}"
    )

# 目标函数
total_cost = 0
for d in range(num_days):
    # 普通车成本
    for k in range(num_truck):
        used = gz[d, k]
        km = gp.quicksum(distance[0, j+1] * visit_truck[d, j, k] for
j in range(num_stores))
        total_cost += truck_cost_day * used + truck_cost_km * km
    # 应急车成本
    for k in range(num_emergency):
        km = gp.quicksum(distance[0, j+1] * visit_emergency[d, j, k]
for j in range(num_stores))
        total_cost += emergency_cost_day * emergency_used[d, k] +
emergency_cost_km * km
    # 腐败成本
    for j in range(num_stores):
        total_cost += decay_cost * (gs[d+1, j] * 0.01)
    # 时间成本
    for k in range(num_truck):
        total_cost += time_cost * (gp.quicksum(distance[0, j+1] /
truck_speed * 60 * visit_truck[d, j, k] for j in range(num_stores)))
    for k in range(num_emergency):
        total_cost += time_cost * (gp.quicksum(distance[0, j+1] /
emergency_speed * 60 * visit_emergency[d, j, k] for j in
range(num_stores)))

m.setObjective(total_cost, GRB.MINIMIZE)
m.optimize()

# 输出

```

```

if m.status == GRB.OPTIMAL or m.status == GRB.SUBOPTIMAL:
    print("每日配送明细: ")
    for d in range(num_days):
        print(f"第{d+1}天: ")
        for j in range(num_stores):
            x_sum = sum(gx[d, j, k].X for k in range(num_truck))
            y_sum = sum(gy[d, j, k].X for k in range(num_emergency))
            print(f"  门店{j+1} 普通车配送量: {x_sum:.2f} 吨, 应急车配
送量: {y_sum:.2f} 吨, 销售: {sales_pred[d, j]:.2f} 吨, 超额:
{over_sales[d, j]:.2f} 吨, 库存: {gs[d+1, j].X:.2f} 吨")

```

5.2 运行结果展示

D:\pythonlearn\venv\Scripts\python.exe D:\pythonlearn\论文复现\物流仿真可运行的.py

Set parameter Username

Set parameter LicenseID to value 2683702

Academic license - for non-commercial use only - expires 2026-07-01

Gurobi Optimizer version 12.0.2 build v12.0.2rc0 (win64 - Windows 11.0 (22631.2))

CPU model: 13th Gen Intel(R) Core(TM) i5-13500H, instruction set [SSE2|AVX|AVX2]

Thread count: 12 physical cores, 16 logical processors, using up to 16 threads

Optimize a model with 793 rows, 425 columns and 1731 nonzeros

Model fingerprint: 0xfda708d8

Variable types: 215 continuous, 210 integer (210 binary)

Coefficient statistics:

Matrix range [1e-02, 1e+03]

Objective range [2e-01, 5e+02]

Bounds range [1e+00, 1e+00]

RHS range [1e-01, 3e+00]

Presolve removed 348 rows and 16 columns

Presolve time: 0.00s

Presolved: 445 rows, 409 columns, 1303 nonzeros

Variable types: 210 continuous, 199 integer (199 binary)

Root relaxation: objective 6.768971e+03, 374 iterations, 0.00 seconds (0.00 work units)

Nodes		Current		Node		Objective	
Bounds		Work					
Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap It/Node
Time							
0	0	6768.97060	0	53	-		
6768.97060	-	-	0s				
H	0	0			8267.8272000		
6768.97060	18.1%	-	0s				
H	0	0			8260.7926000		
6768.97060	18.1%	-	0s				
H	0	0			8248.7414000		
6768.97060	17.9%	-	0s				
H	0	0			7964.3992000		
6768.97060	15.0%	-	0s				
H	0	0			7660.9132000		
6768.97060	11.6%	-	0s				
H	0	0			7259.4366000		
6768.97060	6.76%	-	0s				
H	0	0			6959.4366000		
6768.97060	2.74%	-	0s				
0	0	6780.01752	0	41	6959.43660		
6780.01752	2.58%	-	0s				
H	0	0			6954.4456000		
6780.01752	2.51%	-	0s				
0	0	6781.82931	0	34	6954.44560		
6781.82931	2.48%	-	0s				
0	0	6781.82931	0	36	6954.44560		
6781.82931	2.48%	-	0s				
0	0	6783.14972	0	37	6954.44560		
6783.14972	2.46%	-	0s				
H	0	0			6904.2462000		
6783.14972	1.75%	-	0s				

0	0	6783.33722	0	38	6904.24620
6783.33722	1.75%	- 0s			
0	0	6783.33722	0	39	6904.24620
6783.33722	1.75%	- 0s			
0	0	6783.65184	0	43	6904.24620
6783.65184	1.75%	- 0s			
0	0	6783.82366	0	42	6904.24620
6783.82366	1.74%	- 0s			
0	0	6783.82667	0	43	6904.24620
6783.82667	1.74%	- 0s			
0	0	6783.83054	0	52	6904.24620
6783.83054	1.74%	- 0s			
0	0	6783.83054	0	55	6904.24620
6783.83054	1.74%	- 0s			
0	0	6783.83054	0	53	6904.24620
6783.83054	1.74%	- 0s			
0	0	6783.83054	0	38	6904.24620
6783.83054	1.74%	- 0s			
H	0	0			6903.8462000
6783.83054	1.74%	- 0s			
0	2	6783.83054	0	34	6903.84620
6783.83054	1.74%	- 0s			
H	154	125			6898.5936000
6784.27056	1.66%	25.1 0s			
H	1334	852			6896.2558000
6785.23179	1.61%	19.6 0s			
H	2085	1017			6891.0058000
6786.85458	1.51%	18.3 0s			
H	3125	1342			6890.0900000
6787.90172	1.48%	17.9 0s			
H	3125	1341			6890.0728000
6787.90172	1.48%	17.9 0s			
H	3125	1341			6890.0438000
6787.90172	1.48%	17.9 0s			
H	3125	1334			6889.8796000
6787.90172	1.48%	17.9 0s			
H	3125	1193			6887.6466000
6787.90172	1.45%	17.9 0s			

```

H          3125          1187          6887.5220000
6787.90172  1.45%  17.9    0s
H12570          2245          6887.5058000
6880.52235  0.10%  27.3    3s
  33812          1922          infeasible          33          6887.50580
6884.86934  0.04%  22.8    5s

```

Cutting planes:

```

  Learned: 10
  Gomory: 57
  Cover: 60
  MIR: 187
  Flow cover: 228
  Flow path: 4
  Inf proof: 37
  Network: 16
  Relax-and-lift: 1

```

Explored 38518 nodes (851472 simplex iterations) in 5.40 seconds (5.09 work units)

Thread count was 16 (of 16 available processors)

Solution count 10: 6887.51 6887.51 6887.52 ... 6898.59

Optimal solution found (tolerance 1.00e-04)

Best objective 6.887505800000e+03, best bound 6.887505800000e+03, gap 0.0000%

每日配送明细:

第 1 天:

门店 1 普通车配送量: 1.82 吨, 应急车配送量: 0.00 吨, 销售: 1.09 吨, 超额: 0.00 吨, 库存: 0.73 吨

门店 2 普通车配送量: 1.48 吨, 应急车配送量: 0.00 吨, 销售: 1.01 吨, 超额: 0.00 吨, 库存: 0.47 吨

门店 3 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.92 吨, 超额: 0.00 吨, 库存: 1.08 吨

门店 4 普通车配送量: 1.42 吨, 应急车配送量: 0.00 吨, 销售: 0.83 吨, 超额: 0.00 吨, 库存: 0.59 吨

门店 5 普通车配送量: 1.10 吨, 应急车配送量: 0.00 吨, 销售: 0.77 吨, 超额: 0.00 吨, 库存: 0.33 吨

第 2 天:

门店 1 普通车配送量: 0.83 吨, 应急车配送量: 0.00 吨, 销售: 0.96 吨, 超额: 0.00 吨, 库存: 0.61 吨

门店 2 普通车配送量: 1.65 吨, 应急车配送量: 1.00 吨, 销售: 0.89 吨, 超额: 0.27 吨, 库存: 1.96 吨

门店 3 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.81 吨, 超额: 0.00 吨, 库存: 2.27 吨

门店 4 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.73 吨, 超额: 0.00 吨, 库存: 1.86 吨

门店 5 普通车配送量: 1.52 吨, 应急车配送量: 0.00 吨, 销售: 0.67 吨, 超额: 0.00 吨, 库存: 1.18 吨

第 3 天:

门店 1 普通车配送量: 2.00 吨, 应急车配送量: 0.73 吨, 销售: 0.96 吨, 超额: 0.29 吨, 库存: 2.09 吨

门店 2 普通车配送量: 0.00 吨, 应急车配送量: 0.27 吨, 销售: 0.90 吨, 超额: 0.27 吨, 库存: 1.06 吨

门店 3 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.81 吨, 超额: 0.00 吨, 库存: 1.46 吨

门店 4 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.73 吨, 超额: 0.00 吨, 库存: 1.13 吨

门店 5 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.68 吨, 超额: 0.00 吨, 库存: 0.50 吨

第 4 天:

门店 1 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.99 吨, 超额: 0.00 吨, 库存: 1.10 吨

门店 2 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.91 吨, 超额: 0.00 吨, 库存: 2.15 吨

门店 3 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.84 吨, 超额: 0.00 吨, 库存: 0.62 吨

门店 4 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.75 吨, 超额: 0.00 吨, 库存: 0.38 吨

门店 5 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.69 吨, 超额: 0.00 吨, 库存: 1.81 吨

第 5 天:

门店 1 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 1.00 吨, 超额: 0.00 吨, 库存: 2.10 吨

门店 2 普通车配送量: 0.00 吨, 应急车配送量: 1.00 吨, 销售: 0.93 吨, 超额: 0.28 吨, 库存: 1.94 吨

门店 3 普通车配送量: 2.00 吨, 应急车配送量: 0.00 吨, 销售: 0.85 吨, 超额: 0.00 吨, 库存: 1.77 吨

门店 4 普通车配送量: 1.97 吨, 应急车配送量: 0.00 吨, 销售: 0.76 吨, 超额: 0.00 吨, 库存: 1.59 吨

门店 5 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.70 吨, 超额: 0.00 吨, 库存: 1.11 吨

第 6 天:

门店 1 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 1.07 吨, 超额: 0.00 吨, 库存: 1.03 吨

门店 2 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.95 吨, 超额: 0.00 吨, 库存: 0.99 吨

门店 3 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.90 吨, 超额: 0.00 吨, 库存: 0.87 吨

门店 4 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.81 吨, 超额: 0.00 吨, 库存: 0.78 吨

门店 5 普通车配送量: 0.00 吨, 应急车配送量: 0.58 吨, 销售: 0.75 吨, 超额: 0.22 吨, 库存: 0.72 吨

第 7 天:

门店 1 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 1.03 吨, 超额: 0.00 吨, 库存: 0.00 吨

门店 2 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.99 吨, 超额: 0.00 吨, 库存: 0.00 吨

门店 3 普通车配送量: 0.00 吨, 应急车配送量: 0.26 吨, 销售: 0.87 吨, 超额: 0.26 吨, 库存: 0.00 吨

门店 4 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.78 吨, 超额: 0.00 吨, 库存: 0.00 吨

门店 5 普通车配送量: 0.00 吨, 应急车配送量: 0.00 吨, 销售: 0.72 吨, 超额: 0.00 吨, 库存: 0.00 吨

每日成本分析:

总成本: 6887.51 元

进程已结束, 退出代码为 0

6 参考文献

- [1] ZHENG Hongxing, LIU Baoli, WANG Yan. Dynamic scheduling of multi-type vehicles for fresh agricultural products with perishability constraints [J]. *Transportation Research Part E: Logistics and Transportation Review*, 2023, 178: 103245.
- [2] ARCHETTI C, SAVELSBERGH M W P, SPERANZA M G. Optimization for perishable supply chains [J]. *European Journal of Operational Research*, 2023, 308(2): 509–520.
- [3] LIU Baoli, ZHANG Qiang, ZHENG Hongxing. Robust optimization for cold chain logistics under demand uncertainty: A data-driven approach [J]. *IEEE Transactions on Intelligent Transportation Systems*, 2024, 25(3): 2101–2115.
- [4] SOLOMON M M. Algorithms for the vehicle routing and scheduling problems with time window constraints [J]. *Operations Research*, 1987, 35(2): 254–265.
- [5] ZHANG Delong, LI Xiang, ZHENG Hongxing. Real-time scheduling of emergency vehicles in urban logistics using hybrid simulation-optimization [J]. *Computers & Industrial Engineering*, 2022, 173: 108701.
- [6] BEN-TAL A, GHAOUI L E, NEMIROVSKI A. Robust optimization [M]. Princeton University Press, 2009: 1–28.
- [7] FLEXSIM SOFTWARE PRODUCTS INC. FlexSim simulation modeling software technical reference [Z]. 2025.